MABEL `hardware_bridge`: the Single Hardware-Abstraction Layer

Architecture, safety invariants, and the known drive-velocity gotcha

MABEL Project — Hardware & Control

November 2, 2026

Abstract

`hardware_bridge` is the one process that owns *all* of MABEL's hardware—the mobile base (Teensy MAXSwerve over Ethernet), the DaMiao arms and body, the ORCA hands, the lift, and the cameras. It auto-starts at boot, lives *outside* the ROS colcon workspace, and exposes a generic, `mabel_interfaces`-free DDS contract so that ROS becomes a thin *client* and never touches a device directly. This document records the architecture and the rationale for its key decisions (single owner, where the base inverse kinematics lives, host-side calibration), the safety invariant that exactly one process may command the Teensy (and the oscillation incident that motivated the hard singleton lock), the auto-reconnect and latency properties, and the test suite that validates all of the above in software against a mock Teensy. It closes with the most important gotcha for operators: the base **drive velocity under-reaches the commanded value because the drive SPARK MAX velocity loop ships with `kF=0`**, a defect that the recently added 20 kg payload makes severe—this is a firmware/SPARK tuning item, not a bridge bug, and the document gives both the proper fix and the host-side workaround.

Contents

1 Motivation: why a single hardware owner **2**

2 Architecture **2**
- 2.1 One process, many subsystems — now over the `mabel_hw` library . . . 2
- 2.2 DDS contract (what ROS talks to) . . . 3
- 2.3 Where does the base inverse kinematics live? In the bridge. . . . 3

3 The base: the validated ps5 path **3**

4 Safety: exactly one process may command the Teensy **4**
- 4.1 The oscillation incident . . . 4
- 4.2 The hard singleton lock . . . 4

5 Resilience and latency **4**

6 ROS integration and operation **5**

7 IMPORTANT: the drive-velocity under-reach **5**

8 Nav2 + teleop: the `/mabel_cmd` single-publisher rule **6**

9 Other gotchas **6**

10 Test suite (tools/ + tests/) 7

11 Change log 8

1 Motivation: why a single hardware owner

Before this work the ROS package `mabel_base_hardware` talked to the swerve Teensy *directly*: `swerve_node.py` imported `SwerveClient` and the inverse kinematics from an out-of-repo checkout and opened the UDP link itself. That coupled the ROS launch lifecycle to the motor link, scattered hardware ownership across several ROS nodes, and meant nothing could drive the base unless the full ROS stack was up. Worse, more than one process could open the Teensy link at once, with no guard.

The decision (user-directed, project-wide) is that `~/MABEL/hardware_bridge` is the **single standalone owner of all robot hardware**. ROS hardware packages become thin *bridge clients*; the device protocols never leak past the bridge; and the base can be driven (pad teleop, a CLI test) before ROS is even running. This refines the earlier "server-as-gateway" topology: the on-robot HAL is now this bridge, not a ROS package.

2 Architecture

2.1 One process, many subsystems — now over the `mabel_hw` library

`hardware_bridge.py` is the stable entry point; since the 2026-07 refactor the implementation lives in the installable `mabel_hw` package (see the companion paper `docs/mabel_hw_api.pdf`): the supervisor (`mabel_hw/ros/bridge.py`) opens ONE `MabelHardware` facade and instantiates one DDS adapter *node* per limb (`mabel_hw/ros/*_node.py`), all spun in a single `MultiThreadedExecutor`. The nodes command through per-actuator-group *write leases* and every command topic carries a >1-publisher watchdog; the wire contract below is byte-identical to the pre-refactor bridge (frozen by `tests/contract/contract.json`). Each subsystem owns its device link, degrades gracefully when the hardware is absent (a missing device never crashes the HAL), and is enabled/tuned from `mabel_hw/config.py` or `MABEL_HW_*` environment variables. The registry is:

Subsystem	Device	Transport	Driver source
`base`	3-module MAXSwerve	Teensy, UDP/protobuf	`mabel_hw/drivers/{swerve,maxswerve_host}`
`arms`	DaMiao / OpenArm (14 DoF)	CAN, MIT (default) / Teensy-Ethernet	`mabel_hw/drivers/{damiao_can,teensy_link}`
`body`	DaMiao torso + 3-DoF neck	CAN, MIT (default) / Teensy-Ethernet	`mabel_hw/drivers/` + `kinematics/neck`
`hands`	ORCA hands (Feetech)	**TTL serial**	`mabel_hw/drivers/{orcahand,hands}`
`lift`	BTS7960 DC motor + encoder on a Pico	USB serial	`mabel_hw/drivers/lift` → `firmware/lift/host`
`lidar`	RPLIDAR A2M12	CP2102 serial	`mabel_hw/drivers/rplidar` + `subsystems/lidar`
`cameras`	D435 front + ZED-M head + wrist D405s	native SDK	`mabel_hw/cameras/` (pyrealsense2 / pyzed)

Note that *body* is DaMiao (torso plus a three-motor neck, e.g. DMJ4310) over the *same* CAN/MIT protocol as the arms—the older Dynamixel 2-DoF pan/tilt neck is superseded, so arms and body share `damiao.py`.

Cameras and visual SLAM (2026-07 findings). Three D435 driver behaviours were found to dominate cuVSLAM quality and are now fixed in `cameras/`: (i) the IR *emitter* projects a speckle pattern that is rigid to the camera, so cuVSLAM tracks the dots instead of the world (measured +12 m of z drift handheld) — the driver now defaults the emitter *off* (`CAM_RS_EMITTER`; depth density drops $\approx 80\% \rightarrow 65\%$, an acceptable trade); (ii) a *missing/loose ZED* made `sl.Camera.open()` retry every 3.5 s, each attempt a ∼0.5 s native GIL hold that stalled the D435 IR stream (∼550 ms

gap per retry, and the failing open eventually corrupted the heap — `free(): corrupted unsorted chunks` — crashing the whole HAL); the ZED thread now checks sysfs for the Stereolabs USB VID before ever calling `open`; (iii) images are now stamped with the *sensor-side* (host-clock) librealsense timestamp instead of publish time, so GIL stalls become latency, not false motion. The D435i *IMU* cannot enumerate through the pip pyrealsense2 wheel (V4L2/IIO backend; the Tegra kernel ships no `hid-sensor` modules). The driver therefore opens the motion module *sensor-level on the same device handle* the video pipeline holds and publishes `/camera/front/imu` whenever it enumerates — never from a second process: an external RSUSB grabber taking the device from outside the bridge took the whole camera HAL down (2026-07-01 incident), so the single-owner rule explicitly covers the IMU. To actually light the IMU up, start the bridge with `MABEL_RS_RSUSB=1` (see `run_bridge.sh`): it swaps in the locally built RSUSB-backend pyrealsense2 (`~/librealsense_rsusb`, verified: motion module enumerates, 200 Hz gyro, clean stamps) for the whole process. `mabel_slam`'s `visual_slam.launch.py imu:=auto` then switches cuVSLAM into VIO fusion automatically — and note that once cuVSLAM runs in VIO mode, an IMU-stream interruption stalls its tracker permanently (observed): restart the SLAM launch after any bridge restart.

2.2 DDS contract (what ROS talks to)

The contract is deliberately generic so the bridge depends only on core ROS messages:

Topic	Type	Direction
`/mabel/hw/base/cmd_vel`	`geometry_msgs/Twist`	ROS → bridge (body twist, real frame)
`/mabel/hw/base/estop`	`std_msgs/Bool`	ROS → bridge (latched e-stop)
`/mabel/hw/base/joint_states`	`sensor_msgs/JointState`	bridge → ROS (6 swerve joints)
`/arms/command` ↔ `/arms/joint_states`	`sensor_msgs/JointState`	(same for body)
`/<side>_hand/command` ↔ `.../joint_states`	`sensor_msgs/JointState`	per-side hands
`/lift/command`	`std_msgs/Float64` (m)	ROS → bridge
`/camera/front/*`, `/camera/zed/*`	standard sensor msgs	bridge → ROS (existing SLAM contract)

2.3 Where does the base inverse kinematics live? In the bridge.

We split the two kinematics by data-flow direction:

- **Inverse kinematics** (body twist → per-module drive/steer): *in the bridge.* Commands flow *out* to the actuators, so the IK lives next to the actuator transport. This makes the bridge "`ps5_teleop.py` minus the joystick"—the wire path to the motors is byte-identical whether a PS5 pad, ROS, or a CLI commands it. That code-path identity is the safety argument.
- **Forward kinematics** (wheel encoders → odometry twist): *stays in ROS* (`swerve_odometry.py`). Feedback flows *into* state estimation, so it belongs next to the EKF/SLAM that consumes it; it needs only `/joint_states`, which the bridge publishes.

Consequence: the sim→real 180° yaw flip ($v_x \to -v_x$, $v_y \to -v_y$, $\omega \to \omega$) lives in the ROS client, so the bridge stays in the one canonical real-robot frame ($+x$ forward, $+y$ left) the validated path uses.

3 The base: the validated ps5 path

The entire `maxswerve-host` library is **vendored verbatim** into `base/maxswerve_host/` (the protobuf/UDP `SwerveClient`), so the bridge owns the transport outright. `base/base_ik.py` is the *verbatim* extraction of `ps5_teleop.py`'s incenter geometry `MODULE_POS` (module order `[back, FR, FL]`),

the `inverse_kinematics()` function, and the per-module calibration constants. `base/swerve_base.py` (`SwerveBase`) is the high-level owner: `set_twist` runs the IK and latches the setpoint; a 50 Hz `tick()` re-sends it so the Teensy command watchdog (∼150 ms) stays fed and a dropped command zeros drive (steer held) rather than latching the last speed.

Host-side calibration (decided). The bridge applies `ps5_teleop`'s host-side calibration—`STEER_OFFSET`= $[-0.1, -0.17, 0.1]$ and `DRIVE_DIR`= $[-1, -1, -1]$—before sending (`apply_host_calib=True`). The older ROS `swerve_node` sent *raw* geometry assuming the firmware calibrates; that path is deprecated. **Only one place may apply calibration**: doing it both host-side and in firmware would mis-steer the wheels.

4 Safety: exactly one process may command the Teensy

4.1 The oscillation incident

During bring-up the base oscillated violently back and forth. Root cause: *four* processes were commanding the same Teensy at once—two leaked test bridges, the legacy `server/mabel_swerve_bridge.py` (running under `mabel-bridge.service` with `Restart=always`), and the old direct-Teensy `swerve_node`. They each sent setpoints with their own sequence numbers; the Teensy applied whichever arrived last, so the wheels alternated between conflicting commands. Critically, **ROS-domain isolation does not help**: a second bridge on a different `ROS_DOMAIN_ID` still opens the same UDP socket to the Teensy.

4.2 The hard singleton lock

`hardware_bridge.py` now takes an `flock` on `/tmp/mabel_hardware_bridge.lock` at start-up. The lock is on the *link*, not the DDS graph, so a second instance in any domain fails loudly (`exit 3`) instead of silently double-commanding. `flock` auto-releases when the holder dies, so a crash leaves no stale-lock deadlock; the next bridge acquires cleanly. The legacy `mabel-bridge.service` was stopped and disabled; the app/server now drives the base through ROS (`/mabel_cmd` → client → bridge), never the Teensy directly. `server/real_run.sh`'s `_ensure_bridge` starts the bridge once, idempotently (it checks `systemctl is-active` and `pgrep -f "[h]ardware_bridge.py"`). These guarantees are validated in `tools/singleton_test.sh` (concurrent launch, cross-domain refusal, crash recovery, clean handoff).

5 Resilience and latency

Auto-reconnect. A low-rate watchdog (`SwerveBase.service()`, 2 Hz) detects stale telemetry (Teensy reboot, cable re-plug, or a link that was down at start-up so the keepalive thread died) and tears down/re-opens the `SwerveClient`— leak-safe (a partial connect is closed so its threads do not accumulate). The base thus refreshes and picks the Teensy back up with no relaunch (validated in `tools/reconnect_test.sh`).

Latency. Commands are pushed *immediately* on arrival (the cmd-vel callback calls `tick()` directly), not only on the next 50 Hz stream tick. Measured against a mock Teensy: the bridge hop (`cmd_vel`→IK→UDP) is ∼1.2 ms median; the full `/mabel_cmd`→client→bridge→Teensy path is ∼2.7 ms median.

6 ROS integration and operation

`mabel_base_hardware/swerve_node.py` is now the thin `base_bridge_client`: it subscribes `/mabel_cmd` (`MabelSwerveCmd`, sim frame), applies the 180° flip, and publishes `Twist` to `/mabel/hw/base/cmd_vel`, a `Bool` e-stop, and the lift target to `/lift/command`; it relays the bridge's `/mabel/hw/base/joint_states` onto `/joint_states` for `swerve_odometry`. `server/real_run.sh` starts the bridge (step 0), then launches the ROS stack, and detects the base by waiting for `/odom` to carry data—which only happens once the bridge → client → `swerve_odometry` chain is live. To drive:

```
sudo systemctl start mabel-hardware-bridge      # or: ./server/real_run.sh
ros2 run mabel_teleop keyboard_teleop  # arrows, A/D rotate, SPACE e-stop
```

Keyboard nav speed is adjustable live: `I/K` trim the max linear speed (default 1.2 m/s, step 0.2, up to 4.0 m/s) and `O/L` the max angular speed—useful precisely because of the drive defect below.

7 IMPORTANT: the drive-velocity under-reach

Resolution (the real "won't move" cause): stray competing publishers. Live testing found the base "won't move" was **not the drive at all**—leaked test scripts were publishing `cmd_vel` = 0 on domain 0, drowning out the real commands (the bridge then sent drive ≈ 0). With a *single* publisher the loaded base (20 kg) drives cleanly at every speed: cmd 1.0 → 0.34, 0.5 → 0.25, 0.4 → 0.21, 0.3 → 0.16 m/s. The bridge sends byte-identical `setCommand` to `ps5_teleop` and behaves identically—**there is no bridge bug and no hidden conversion**. *Lesson:* the singleton `flock` guards two bridge *processes*, but not two ROS nodes publishing the same command topic; ensure exactly one publisher on `/mabel/hw/base/cmd_vel` and never leave test publishers running. The `kF=0` under-reach and the high-speed collapse below are still real, just not the cause of "won't move".

Symptom (as first reported). Steering tracks beautifully, but the drive wheels barely turn / will not spin.

It is *not* a bridge bug. The host commands velocity faithfully: verified against a mock Teensy, a commanded value arrives unchanged and a direct `cmd_vel` passes through 1:1 with no cap. The bridge sends byte-identical `setCommand` values to `ps5_teleop`.

Root cause (measured on the real base, wheels off the ground). `tools/drive_diag.py` commanded a forward velocity ladder and read back the encoder velocity, duty, and current per module:

commanded	actual wheel speed	tracked	drive current (unloaded)
0.5 m/s	∼0.26 m/s	∼52 %	7 A to 13 A
1.0 m/s	∼0.51 m/s	∼51 %	17 A to 21 A
2.0 m/s	[0.03, 0.54, 0.01] **(collapses)**	—	17 A to 21 A

Two defects, both on the SPARK side (the documented `kF=0`): (i) the loop *under-reaches to* ∼50 % of the commanded m/s, even unloaded—so it is **not a load/torque problem**; and (ii) above ∼1.5 m/s commanded the loop **collapses**—two of three wheels drop to ∼0 while drawing 17 A to 21 A. **Commanding faster therefore makes it stall, not go faster.** The 17 A to 21 A draw on *free-spinning* wheels also points to heavy drivetrain drag and/or an aggressively mistuned loop.

Proper fix (firmware / REV Hardware Client). On each drive SPARK MAX: set the velocity `kFF` (kills the ∼50 % under-reach), review the smart current limit and closed-loop ramp (the collapse looks current-limited / unstable), and check the idle mode and drivetrain for the abnormal free-spin current. This is the only real fix.

There is no safe host-side workaround. `config.py` keeps a `drive_gain` knob, but because the loop *collapses* above ∼1.5 m/s, a gain > 1 (or raising the keyboard speed cap) pushes the drive *into* the collapse and makes it worse. Until the SPARK is tuned the usable command range is roughly 0.5 m/s to 1.2 m/s (giving ∼0.25 m/s to 0.6 m/s actual); the keyboard caps were set accordingly (`MAX_LIN_SPEED` = 1.0 m/s, `LIN_SPEED_MAX` = 1.3 m/s). Use `tools/drive_diag.py` (wheels off the ground) to re-measure after tuning.

8 Nav2 + teleop: the `/mabel_cmd` single-publisher rule

Only ONE node may publish `/mabel_cmd` (and `/mabel/hw/base/cmd_vel`) at a time. Two publishers do not "merge" – they interleave on the subscriber, and the more frequent one wins. This caused the base to STALL under keyboard teleop twice, even though every individual component was correct:

- Leaked test scripts publishing `cmd_vel`=0 at high rate drowned every command.
- `mabel_navigation/cmd_vel_bridge.py` **(started by Nav2) published `/mabel_cmd` at 50 Hz *unconditionally* – including ZEROS while Nav2 was idle.** Run alongside the keyboard (which publishes `/mabel_cmd` at 20 Hz), the 50 Hz idle zeros won, so the drive sat at ≈0 and the base "would not move".

Fix applied: `cmd_vel_bridge._step` now stays SILENT when idle – it publishes `/mabel_cmd` only while Nav2 is actively commanding (a `/cmd_vel` within `cmd_timeout`) OR still ramping to a stop ($|v| > 10^{-3}$). When idle it emits nothing, so teleop owns `/mabel_cmd` and the hardware-bridge watchdog holds the base at zero. Verified live (2026-06-28): with the fix, keyboard teleop drives the loaded base back and forth at 0.21 m/s *with `cmd_vel_bridge` running*.

FUTURE Nav2-on-the-real-robot warning. If the base stalls under teleop while Nav2 is up, the first check is `ros2 topic info /mabel_cmd`: more than one publisher means they are fighting. The "silent when idle" fix handles teleop XOR nav, but NOT teleop and an active nav goal at the same time. The robust long-term answer is a single *arbiter* node (priority estop > teleop > nav) that is the SOLE `/mabel_cmd` publisher, with the keyboard and Nav2's `cmd_vel_bridge` feeding it on *separate* input topics (e.g. `/teleop/cmd` and `/nav/cmd`) – so they can never interleave on the base command. (A server-side arbiter of this shape already exists; mirror it on the ROS side before relying on Nav2 + teleop concurrently.)

9 Other gotchas

- **NO-CARRIER base link.** The base Teensy is reached over Thor's 100G `QSFP28` break-out cable, which fans out into four `nvethernet` 10 GbE lanes `mgbe0_0` … `mgbe3_0`. The base lives on `mgbe3_0`, with the host IP `192.168.124.2/24` pinned by a persistent NetworkManager profile `teensy-base` (`ipv4.never-default` so it cannot steal the wifi/Tailscale default route; `autoconnect` on, so it comes back after a reboot). The Teensy is the static peer `192.168.124.10:5800` (UDP/protobuf). If `ethtool mgbe3_0` says "Link detected: no", the break-out cable is unplugged or the Teensy is unpowered; if the link is up but `192.168.124.2` is missing, reassert it with `nmcli connection up teensy-base`. `real_run.sh` now preflights

both—it re-activates the profile before warning—and the bridge auto-reconnects once the link returns. *(The old Realtek r8126 NIC enP2p1s0 is retired; do not use it.)*

- **Adding more Teensys (arms / body) on the break-out.** The other three lanes are free for additional Teensys. Give each its own `/24` on a distinct subnet and a matching `nmcli` static profile, mirroring `teensy-base`. Suggested convention:

Lane	Subsystem	Host IP	Teensy IP
`mgbe3_0`	base (live)	`192.168.124.2/24`	`192.168.124.10`
`mgbe2_0`	arms	`192.168.125.2/24`	`192.168.125.10`
`mgbe1_0`	body / torso	`192.168.126.2/24`	`192.168.126.10`
`mgbe0_0`	spare	`192.168.127.2/24`	`192.168.127.10`

Create one like the base with, e.g. (arms):

```
sudo nmcli connection add type ethernet con-name teensy-arms ifname mgbe2_0 \
  ipv4.method manual ipv4.addresses 192.168.125.2/24 ipv4.never-default yes \
  ipv6.method link-local connection.autoconnect yes
```

Keep separate subnets per lane so the lanes never collide and each Teensy's IP encodes which limb it drives.

- **Never `pkill` a bridge by name.** All test scripts in `tools/` guard against running while a real bridge is up, and clean up only their own PIDs; a `pkill -f hardware_bridge.py` would take down the production bridge.
- **Single base owner is enforced, but the flock is voluntary.** The bridge takes an exclusive `flock` on `/tmp/mabel_hardware_bridge.lock` at startup, so a SECOND `hardware_bridge.py` exits 3 ("a second owner would make the base oscillate"). But that only stops a second *bridge* — the lock is advisory, so a NON-bridge talker (`ps5_teleop.py`, a `tools/live_*_diag` probe, an old out-of-repo direct bridge) sends straight to `192.168.124.10:5800` and fights the bridge, which reads as the base "stalling sometimes". `real_run.sh` now closes this: `_reap_stale` kills those stray commanders on every launch, and `_verify_single_base_owner` (run right after `_ensure_bridge`) asserts exactly one bridge holds the lock, kills any duplicate (keeping the lock holder), and prints "single base owner confirmed". The ROS nodes `swerve_node`/`cmd_vel_bridge`/`sim_teleop_bridge` are command *sources* feeding the one bridge over `/mabel_cmd`, NOT Teensy talkers (`cmd_vel_bridge` stays silent when idle so it cannot fight teleop).
- **Python deps** live in the conda `mabel` (RoboStack ROS 2) env, which the bridge must run under for DDS interop: `numpy`, `protobuf` (base); `python-can` (arms/body; *not* installed by default); `pyserial` (hands/lift); `pyrealsense2`, `pyzed` (cameras).

10 Test suite (`tools/` + `tests/`)

Since the `mabel_hw` refactor the bridge also carries a pytest tree (`tests/`, 156 tests, hardware-free): imports-compat + shim identity (every pre-refactor import path must hand out the same objects), the frozen contract snapshot (topics/ports/env/spawn strings vs `tests/contract/contract.json`), lease arbitration, the calibration store, per-driver unit tests against wire-exact fakes, the full facade on mock plants, and an AST structure lint of the ROS nodes. Run `python3 -m pytest tests -q` anywhere; `ros`-marked tests auto-skip without `rclpy` and execute on thor. The shell tools below are unchanged and remain the ROS-level end-to-end gate (**always bracket `software_selftest.sh` with**

`sudo systemctl stop/start mabel-hardware-bridge` — **its cleanup kills any running bridge**). All run on `127.0.0.1` against a mock Teensy and never touch the real robot:

Tool	What it proves
`fake_teensy.py`	mock MAXSwerve Teensy (exact protobuf/UDP wire protocol)
`software_selftest.sh`	two-way comms, `/odom` detection, e-stop, stability (9 checks)
`singleton_test.sh`	single-owner under concurrency, cross-domain, crash, handoff (8 checks)
`reconnect_test.sh`	disconnect → reconnect auto-recovery (6 checks)
`keyboard_teleop_test.py`	keystrokes → `/mabel_cmd` → bridge → mock (6 checks)
`latency_test.py`	command-path latency, bridge-only and full ROS path
`drive_speed_test.py`	velocity is commanded faithfully and scales with I/K
`drive_diag.py`	*real-base* commanded-vs-actual velocity + current (drive bug)
`handshake_seq_test.py`	command seq is unique + ordered + exception-free under concurrent senders

11 Change log

2026-07-11 — the `mabel_hw` refactor. The HAL became an installable library (`pip install -e .`): typed facade (`MabelHardware`, `read()`/`claim()`/`estop()`), per-actuator-group write leases, `calibration_data/` as the dual-written read-first calibration mirror, Teensy-Ethernet arm/neck backends (vendored upstream `robot_host`, off by default) beside the proven CAN path, the RSSR neck kinematics with task-space `set_gaze`, and the DDS nodes rewired as thin adapters — with every pre-refactor import path, topic, port, and spawn string preserved by shims and frozen in tests. Details, measured overheads, and the experiment harness: `docs/mabel_hw_api.pdf`.

Original bring-up (2026-06):

1. Created `hardware_bridge/` as the single HAL: vendored `maxswerve_host`, extracted ps5 IK + calibration + joystick, `SwerveBase`, per-limb subsystems (`base`/`arms`/`body`/`hands`/`lift`), native-SDK `cameras/`, the `hardware_bridge.py` supervisor, `config.py`, `paths.py`.
2. Systemd auto-start (`install/mabel-hardware-bridge.service`) + headless installer; `run_bridge.sh` sources the conda ROS env for DDS interop.
3. Revised `mabel_base_hardware` into the `base_bridge_client`; updated `base.launch.py`, `setup.py`, `package.xml`.
4. Wired `server/real_run.sh`: `_ensure_bridge` (idempotent) + a base-link preflight; fixed the bridge-detection `pgrep` pattern.
5. Hard singleton lock; retired the legacy `mabel-bridge.service`.
6. Auto-reconnect (`SwerveBase.service()`); immediate-send latency cut.
7. Keyboard teleop: raised default linear speed 0.8→1.2 m/s, widened the live `I/K/O/L` range.
8. Diagnosed the drive under-reach (SPARK `kF=0` + 20 kg); added the `drive_gain` workaround and `drive_diag.py`.
9. Added the full `tools/` test suite (all green in software) and hardened every test against killing a real bridge.
10. **Fixed a command-handshake thread-safety bug in `SwerveClient`.** The per-command `seq` (echoed by the firmware, which ignores any command not newer than the last applied) was bumped and sent non-atomically, and the RTT dict was shared with the RX thread unlocked. With ≥ 2 senders (ROS `cmd_vel` + pad teleop + health poller + keepalive) commands could reach the Teensy out of order / with a reused seq ⇒ the firmware drops the loser and that module holds its last setpoint = an intermittent single-wheel *stall*. Now the "bump seq → serialize → `sendto`" section is serialized by a reentrant send lock and the RTT bookkeeping

has its own lock. `handshake_seq_test.py` regresses it: 0 duplicate / 0 out-of-order over 3000 concurrent commands (was 36 out-of-order before the fix). NB: under the current single-mutually-exclusive-callback-group bridge there is effectively one sender, so this fires only once a second sender exists — but it is the exact "handshake-byte" failure mode and is now closed. A persistent single-wheel stall with one sender is more likely the SPARK `kF=0` stiction (see the drive-under-reach note), not the host.

11. **Side-aware hand auto-detection.** The two ORCA hand USB adapters are identical CH340s, so first-come-first-served detection could hand the left adapter to the right hand. Hand ports now default to `auto` and resolve through a serial-number→side registry (`config.HANDS[adapter_serials]`, right = `5AE6024091`; `detect_side_port` in `orcahand/ports.py`): a side connects only to *its* registered adapter; a side with no registered serial may claim the single unclaimed adapter (logged with the serial to pin); ambiguity resolves to idle + the 2 s hot-plug watchdog. `server/real_run.sh` preflights the same serials to report which hand is present and resolve `--hands auto`, and its status watchdog now shows per-side *live-feedback* indicators (`handL`/`handR`). Idle-retry log lines are deduplicated (logged once per cause change, not every 2 s retry).

Open item. The drive under-reach (§7) is not yet resolved: it needs the SPARK `kFF`/current-limit tuned, *or* `drive_gain` fitted from a `drive_diag.py` run on the real base. Everything else is implemented and validated in software.